



Proyecto Final

05.2026.IC.90501.G1.TÉCNICAS DE
PROGRAMACIÓN

Stephanie Solis

Dragon Nutrex

Nutrición para todos

Universidad Politecnica

Documentación Técnica: Dragon Nutrex	3
1. Introducción y Alcance del Proyecto	3
Alcance Detallado	3
2. Especificación de Requerimientos	4
3. Tech Stack	4
4. Arquitectura y Patrones de Diseño	5
4.1 Patrón Singleton (Gestión de Conexión Redis)	5
4.2 Patrón Repository (Repositorio)	5
4.3 Patrón MVC (Modelo-Vista-Controlador)	5
5. Arquitectura de Datos: Deep Dive en Redis	6
5.1 Justificación Técnica	6
5.2 Estructura de Almacenamiento	6
5.3 Patrón Repository	7
6. Aseguramiento de Calidad (QA) y Cumplimiento	7
6.1 Validación de Integridad y Lógica	7
6.2 Pruebas de Integración (Redis)	7
6.3 Seguridad y Cumplimiento Legal	8
7. Historial de Versiones	8
8. Manuales de usuario	8
8.1 Manual del Administrador (Perfil Admin)	8
A. Gestión de Usuarios y Directorio Maestro	8
B. Mantenimiento del Catálogo de Productos	9
C. Módulo de Estadísticas de Consumo	9
D. Módulo de Estadísticas Globales	10
E. Cómo Exportar Datos (Procedimiento Único de Reportes)	10
8.2 Manual del Usuario (Perfil Estándar)	11
A. Navegación y Perfil Antropométrico	11
B. Gestión de Menús Diarios	11
C. Visualización de Progreso (Estadísticas)	12
8.3 Manual Técnico de Implementación: Dragon Nutrex	13
1. Perspectiva del Desarrollador	13
2. Prerrequisitos del Entorno	13
3. Guía de Implementación Paso a Paso (A to Z)	14
Fase 1: Aprovisionamiento de Infraestructura	14
Fase 2: Configuración de la Capa de Datos	14
Fase 3: Inicialización de Datos (The Seeding Script)	14
Fase 4: Compilación y Despliegue	14
4. Conclusión Técnica	15
9. Conclusión Técnica: Excelencia en el Diseño y Persistencia	15
¿Por qué esta arquitectura es la decisión correcta?	15
El Valor del Proceso	16

Documentación Técnica: Dragon Nutrex

Proyecto: Nutrición para Todos

Versión: 1.2.0 (Final)

Responsable: Stephanie Solís

1. Introducción y Alcance del Proyecto

Dragon Nutrex es una plataforma integral para la gestión nutricional que ha evolucionado de un entorno local a una arquitectura web robusta basada en .NET 8. El sistema permite a los usuarios finales tomar control de su salud mediante el seguimiento de datos antropométricos y la planificación de menús, mientras otorga al administrador herramientas de supervisión y mantenimiento global.

Alcance Detallado

- **Gestión de Perfiles y Datos Antropométricos (Ambos Roles):** El sistema permite capturar y actualizar medidas físicas (peso, altura, edad, nivel de actividad). Esta información es la base para el cálculo dinámico del IMC y las tasas metabólicas.
- **Planificación Operativa de Menús (Ambos Roles):** Implementación completa de funciones CRUD (Crear, Leer, Actualizar, Borrar) para menús diarios. Tanto el usuario como el administrador pueden diseñar planes alimenticios vinculados a la base de datos de productos.
- **Directorio Maestro y Seguridad (Exclusivo Admin):** Control total de cuentas de usuario, permitiendo la depuración de registros y la gestión de permisos.
- **Mantenimiento de Catálogo (Exclusivo Admin):** Gestión del inventario de productos alimenticios y sus densidades calóricas/macronutrientes.
- **Reportes y Exportación (Exclusivo Admin):** Capacidad para extraer listados de usuarios en formatos CSV, TXT y PDF.
- **Análisis Visual: Panel interactivo de "Mi Progreso"** que procesa datos históricos para mostrar la evolución del usuario.

2. Especificación de Requerimientos

ID	Requerimiento Funcional	Prioridad	Perfil
RF-01	Autenticación con cifrado BCrypt de alta seguridad.	Alta	Ambos
RF-02	CRUD completo de información antropométrica.	Alta	Ambos
RF-03	CRUD completo de Menús diarios y planes.	Alta	Ambos
RF-04	Gestión centralizada de catálogo de Productos.	Alta	Admin
RF-05	Exportación de datos a PDF, CSV y TXT.	Media	Admin

3. Tech Stack

El proyecto utiliza un stack de última generación enfocado en el rendimiento y la escalabilidad:

- Frontend & Backend: ASP.NET Core Blazor Web App (.NET 8). Se utiliza el modo InteractiveServer para permitir una comunicación bidireccional en tiempo real mediante SignalR, eliminando la necesidad de refrescar la página.

- Persistencia de Datos (Core): Redis. Motor de estructura de datos en memoria que funciona como base de datos NoSQL de baja latencia.
- Lenguaje de Programación: C# bajo estándares de Clean Code y principios SOLID.
- Seguridad: Librería BCrypt.Net-Next para el hashing de contraseñas.
- Estilos y UI: Bootstrap 5 para un diseño responsivo y moderno.
- Entorno de Desarrollo: VS Code / Visual Studio sobre macOS.

4. Arquitectura y Patrones de Diseño

El sistema garantiza su estabilidad y rendimiento mediante la aplicación de patrones de software específicos:

4.1 Patrón Singleton (Gestión de Conexión Redis)

Se utiliza para administrar el objeto `ConnectionMultiplexer` de Redis. Dado que abrir y cerrar conexiones es costoso en términos de recursos, se mantiene una única instancia activa durante toda la vida de la aplicación. Esto asegura una comunicación ultra rápida y eficiente con la base de datos y evita el agotamiento de recursos del servidor.

4.2 Patrón Repository (Repositorio)

Se implementó mediante la interfaz genérica `IRepository<T>`.

- Propósito: Este patrón actúa como un mediador entre los Servicios y Redis. Los servicios solicitan datos sin conocer la complejidad interna de las llaves de Redis.
- Beneficio: Facilita el mantenimiento y las pruebas unitarias. Si en el futuro se decide migrar a una base de datos relacional (SQL), solo se debe crear una nueva implementación del Repositorio sin alterar el resto del código.

4.3 Patrón MVC (Modelo-Vista-Controlador)

Organización clara del código dividiendo la representación de datos (Modelos), la interfaz interactiva (Vistas Razor) y la lógica de flujo (Controladores).

5. Arquitectura de Datos: Deep Dive en Redis

La decisión técnica más crítica del proyecto fue la migración de archivos JSON a Redis, un motor de estructura de datos en memoria que funciona como base de datos NoSQL de baja latencia.

5.1 Justificación Técnica

Redis fue seleccionado para Dragon Nutrex por tres razones principales:

1. **Velocidad Extrema:** Al residir en memoria RAM, las operaciones de lectura/escritura son órdenes de magnitud más rápidas que las bases de datos relacionales tradicionales.
2. **Persistencia Flexible:** Permite guardar objetos complejos como Hashes, lo que facilita el mapeo directo de nuestras clases de C# (`Usuario`, `Producto`, `Menu`).
3. **Escalabilidad:** Maneja eficientemente múltiples conexiones concurrentes, ideal para un sistema que requiere actualizaciones constantes de progreso.

5.2 Estructura de Almacenamiento

Se implementó un sistema de llaves jerárquicas para organizar la información:

- **Usuarios:** `usuario:{Guid}` (Almacenado como Hash para acceso rápido por ID).
- **Índices de Correo:** `usuario:email:{email}` (Llave secundaria para permitir el login rápido sin recorrer toda la base de datos).
- **Menús:** `menu:{usuarioId}:{fecha}` (Permite organizar la dieta diaria de forma relacional dentro de un entorno NoSQL).

5.3 Patrón Repository

Para interactuar con Redis, el proyecto utiliza una interfaz genérica `IRepository<T>`, lo que desacopla la lógica de negocio del motor de datos. Esto permite que el `UsuarioService` realice operaciones asíncronas (`Task<T>`) optimizando el uso de recursos del servidor y facilitando la mantenibilidad a largo plazo.

6. Aseguramiento de Calidad (QA) y Cumplimiento

El proceso de QA se enfocó en garantizar la integridad de los cálculos nutricionales y la estabilidad de la conexión con la base de datos distribuida.

6.1 Validación de Integridad y Lógica

- **Filtros de Controlador:** El controlador implementa validaciones de entrada que verifican la existencia de IDs y nombres antes de renderizar la interfaz. Esto evita excepciones por duplicidad de llaves (**Duplicate Key**) o fallos de renderizado ante registros incompletos en Redis.
- **Pruebas de Cálculo:** Se validaron manualmente las fórmulas de IMC y Tasa Metabólica Basal comparando los resultados del sistema contra cálculos externos para asegurar la precisión de los servicios de salud.

6.2 Pruebas de Integración (Redis)

- **Persistencia en Tiempo Real:** Se realizaron pruebas de "estrés" cerrando y abriendo la sesión para verificar que el Patrón Singleton mantuviera la conexión con Redis estable y que los datos antropométricos se persistieran correctamente sin pérdida de información entre cambios de vista.
- **Manejo de Nulos:** Se testearon escenarios donde el usuario no tiene historial previo (usuarios nuevos), asegurando que la aplicación muestre estados de "Carga" o "Sin datos" en lugar de fallar por punteros nulos.

6.3 Seguridad y Cumplimiento Legal

- **Protección de Datos:** El diseño cumple con la Ley N° 8968 (Costa Rica). La información sensible del usuario está protegida mediante el hashing de contraseñas con BCrypt, asegurando privacidad y cumplimiento con estándares de encriptación.

7. Historial de Versiones

- v1.0.0 (14/04/2026): MVP inicial con persistencia en JSON.
- v1.1.0 (20/04/2026): Migración a Blazor Web App y conexión con Redis.
- v1.2.0 (27/04/2026): Implementación de roles Admin/Usuario, exportación exclusiva y optimización de carga asíncrona.

8. Manuales de usuario

8.1 Manual del Administrador (Perfil Admin)

A. Gestión de Usuarios y Directorio Maestro

1. En el menú lateral, haga clic en "Usuarios".
2. Revise la lista de registros activos. Para modificar un usuario, seleccione el botón de edición; para eliminarlo, use el ícono de papelera.
3. Para cambiar o restablecer el password del usuario, seleccione "Reset Password" en amarillo.
4. Confirme cualquier cambio para que se actualice instantáneamente en Redis.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: Administrador



Usuarios

Productos

Menús

Estadísticas

Estadísticas de Consumo

Mantenimiento de Usuarios

Exportar vista actual: [CSV](#) [TXT](#) [PDF](#)

 Buscar usuario por nombre o correo...

Nombre	Correo	Dieta	Estado	Acciones
adrian	adrian@adrian.com	Keto	Activo	Desactivar Reset Pwd 
alejandro	alejandro@alejandro.com	Vegetariana	Activo	Desactivar Reset Pwd 
andrea	andrea@andrea.com	Vegetariana	Activo	Desactivar Reset Pwd 

B. Mantenimiento del Catálogo de Productos

1. Navegue al módulo "Catálogo de Alimentos" o "Productos".
2. Utilice el botón "Nuevo Producto" para introducir alimentos con su valor calórico y nutricional.
3. Para editar productos existentes, localice el registro en la tabla y actualice los campos. Puede buscar el producto en la barra de búsqueda también.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: Administrador

Usuarios Productos Menús Estadísticas Estadísticas de Consumo

Gestión de Productos

Exportar vista actual: CSV TXT PDF

Nombre del producto
Ej: Atún en agua

Calorías (kcal) 0

Proteínas (g) 0

Carbohidratos (g) 0

Grasas (g) 0

GUARDAR PRODUCTO NUEVO LIMPIAR / NUEVO

Buscar producto en la tabla...

Nombre	Calorías	Proteínas	Carbos	Grasas
Pollo 1	82,0	23,4g	1,8g	20,5g
Arroz 2	195,0	21,4g	34,0g	14,1g

C. Módulo de Estadísticas de Consumo

1. Ingrese a "Estadísticas de Consumo".
2. Visualice los productos más utilizados en los menús de todos los usuarios y el promedio de calorías consumidas por la comunidad.
3. Utilice estos datos para identificar tendencias alimenticias o productos que requieran actualización de información nutricional.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: Administrador

Usuarios Productos Menús Estadísticas Estadísticas de Consumo

Seleccionar Usuario
carolina

Fecha Inicio
28/03/2026

Fecha Fin
27/04/2026

Consultar Historial

Exportar vista actual: CSV TXT PDF

Seguimiento de Dieta

✓ Datos cargados: 8 días encontrados para carolina.

Día (Click para ver detalle)	Consumo (kcal)	Objetivo (kcal)	Estado
25/04/2026	1,014,0	2,000,0	Adecuado
24/04/2026	9,259,0	2,000,0	Exceso
20/04/2026	392,0	2,000,0	Adecuado

D. Módulo de Estadísticas Globales

1. Seleccione "Estadísticas Generales" en el panel principal.
2. Observe el resumen de la base de usuarios: productos más consumidos, total de registros, distribución por niveles de actividad y promedios de IMC globales.
3. Este módulo procesa todos los Hashes de la base de datos para dar una visión macro del estado de salud de la población registrada.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: Administrador

Usuarios Productos Menús Estadísticas Estadísticas de Consumo

Estadísticas de Consumo Globales

Exportar vista actual: CSV TXT PDF

Fecha Inicio: 28/03/2026 Fecha Fin: 27/04/2026 Generar Reporte

Top 5 Productos Más Consumidos		Porcentaje por Tipo de Dieta	
Atun 6	233,0 consumidos	Vegetariana	28,6%
Jamon Serrano	123 consumidos	Vegana	25,7%

E. Cómo Exportar Datos (Procedimiento Único de Reportes)

Nota: Esta facultad es exclusiva del administrador y está disponible en los módulos con tablas de datos.

1. Localice el panel de "Exportación" sobre la tabla principal.
2. Seleccione el formato deseado: CSV, TXT o PDF.
3. El sistema procesará los datos y descargará el archivo generado automáticamente.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: Administrador

Usuarios Productos Menús Estadísticas Estadísticas de Consumo

Seleccionar Usuario: daniel Fecha Inicio: 28/03/2026 Fecha Fin: 27/04/2026 Consultar Historial

Seguimiento de Dieta

Exportar vista actual: CSV TXT PDF

✓ Datos cargados: 9 días encontrados para daniel.

Día (Click para ver detalle)	Consumo (kcal)	Objetivo (kcal)	Estado
16/04/2026	110,0	2.000,0	Adecuado
16/04/2026	340,0	2.000,0	Adecuado

8.2 Manual del Usuario (Perfil Estándar)

A. Navegación y Perfil Antropométrico

1. Al iniciar sesión, si necesita modificar su información antropométrica navegue a la sección "Mi Perfil" y presione "Editar perfil"
2. Modifique sus datos físicos (peso, altura, actividad).
3. Presione "Guardar"; verá su IMC actualizado de inmediato sin recargar la página.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: flora@flora.com

Mis Menús **Mi Progreso (Estadísticas)**

Usuario: Flora
Dieta: Normal

Peso: 60,0 kg
Actividad: Sedentaria

Altura: 175 cm
Objetivo: Bajar peso

Tu IMC
19,6 - Peso normal

[Editar Perfil](#)

B. Gestión de Menús Diarios

1. Seleccione la opción "Mis Menús" en la barra de navegación.
2. Use el calendario para elegir el día y añada alimentos del catálogo.
3. El sistema sumará automáticamente las calorías del día, así como cualquier cambio en menús existentes.

Usuario: Flora
Dieta: Normal

Peso: 60,0 kg
Actividad: Sedentaria

Altura: 175 cm
Objetivo: Bajar peso

Tu IMC
19,6 - Peso normal

[Editar Perfil](#)

Mi Diario de Comidas [GUARDAR CAMBIOS](#)

Mi Perfil
Flora (flora@flora.com)

Fecha del Menú
27/04/2026

[Nuevo Día](#)

Seleccionar Producto
-- Escoger alimento --

Cantidad (Porciones)
1

[+ Agregar](#)

Menú del: 27/04/2026

No has agregado alimentos para este día.

Historial de Menús Registrados

Fecha	Calorías	Alimentos Principales	Acciones
26/04/2026	7.265 kcal	Huevo 3, Manzana 90, Jamon Serrano...	Editar Borrar

- Si lo desea, puede hacer click en cada menú para editarlo, la información se actualizará automáticamente, una vez guardados los cambios.

Menú del: 27/04/2026

Producto	
Arroz 2	
Pollo 51	
Huevo 13	
Pasta 27	

Menú del 27/04/2026

Alimento	Cant.	Calorías	Proteína	Carbos	Grasas
Arroz 2	1	195,0	21,4 g	34,0 g	14,1 g
Pollo 51	10	790,0	280,0 g	240,0 g	272,0 g
Huevo 13	15	1.320,0	676,5 g	628,5 g	48,0 g
Pasta 27	12	3.972,0	151,2 g	714,0 g	327,6 g
TOTALES DEL DÍA:		6.277,0	1.129,1 g	1.616,5 g	661,7 g

Editar este día
Cerrar

Quitar

Historial de Menús Registrados

Fecha	Calorías	Alimentos Principales	Acciones
27/04/2026	6.277 kcal	Arroz 2, Pollo 51, Huevo 13...	Editar Borrar

Cerrar sesión

C. Visualización de Progreso (Estadísticas)

- Navegue a la sección "Estadísticas" o "Mi Progreso".
- Observe las gráficas de evolución generadas a partir de sus datos históricos guardados en la aplicación.

Seleccione un módulo para agregar, visualizar, editar o eliminar datos y realizar cálculos.
Sesión activa: flora@flora.com

Mis Menús
Mi Progreso (Estadísticas)

Usuario: Flora
Dieta: Normal

Peso: 60,0 kg
Actividad: Sedentaria

Altura: 175 cm
Objetivo: Bajar peso

Tu IMC
19,6 - Peso normal

Editar Perfil

Seguimiento de Dieta

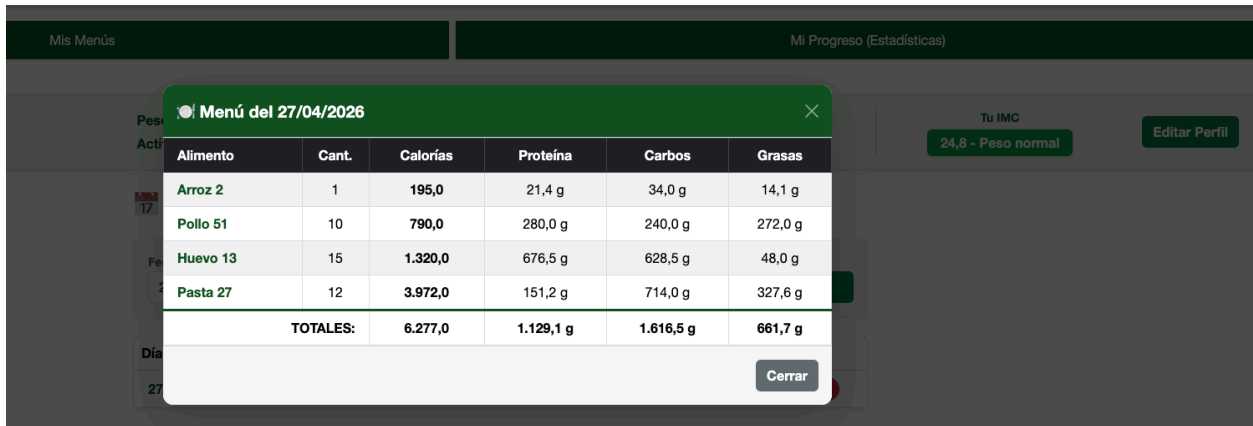
Fecha Inicio
28/03/2026

Fecha Fin
27/04/2026

Consultar Historial

Día (Click para ver detalle)	Consumo (kcal)	Objetivo (kcal)	Estado
26/04/2026	7.255,0	2.000,0	Exceso

3. Si desea conocer el detalle del menú, puede hacer clic sobre el menú



Alimento	Cant.	Calorías	Proteína	Carbos	Grasas
Arroz 2	1	195,0	21,4 g	34,0 g	14,1 g
Pollo 51	10	790,0	280,0 g	240,0 g	272,0 g
Huevo 13	15	1.320,0	676,5 g	628,5 g	48,0 g
Pasta 27	12	3.972,0	151,2 g	714,0 g	327,6 g
TOTALES:		6.277,0	1.129,1 g	1.616,5 g	661,7 g

8.3 Manual Técnico de Implementación: Dragon Nutrex

Audiencia: Desarrolladores de Software / Ingeniería de Sistemas

Arquitectura: .NET 8 + Redis Cloud + MVC Pattern

1. Perspectiva del Desarrollador

El proyecto Dragon Nutrex ha sido diseñado bajo principios de alta cohesión y bajo acoplamiento. Como desarrolladores, hemos implementado una arquitectura donde la persistencia NoSQL no es solo un almacenamiento, sino un motor de alta velocidad que soporta la lógica nutricional en tiempo real. La elección de Redis responde a la necesidad de latencias ultra bajas, mientras que el patrón Repository asegura que el código sea testeable y mantenible a largo plazo.

2. Prerrequisitos del Entorno

Para garantizar una implementación determinista, el host debe cumplir con:

- .NET 8.0 SDK (LTS): Framework de ejecución principal.
- Redis Engine 7.2+: Soporte para estructuras HASH y SET.
- Python 3.10+: Para scripts de automatización de datos (Seeding).
- Docker Desktop: Para orquestación de servicios locales.
- Redis Insight: Herramienta de auditoría visual de memoria.

3. Guía de Implementación Paso a Paso (A to Z)

Fase 1: Aprovisionamiento de Infraestructura

Clone el repositorio oficial y levante la instancia de base de datos:

```
git clone [https://github.com/BeatrixxK/Codigo_Entregable_1.git]
docker run -d --name dragon-redis -p 6379:6379 redis/redis-stack:latest
```

Fase 2: Configuración de la Capa de Datos

Edite el archivo appsettings.json para establecer el handshake con Redis. El uso del patrón Singleton en el ConnectionMultiplexer es obligatorio para optimizar los sockets del servidor.

```
"ConnectionStrings": {
  "RedisConnection": "localhost:6379,password=tu_clave,abortConnect=false"
}
```

Fase 3: Inicialización de Datos (The Seeding Script)

Ejecute el script de Python para inyectar el catálogo maestro de productos en Redis. Esto mapea objetos directamente a la RAM mediante comandos HSET.

```
pip install redis
python seed_data.py
```

Fase 4: Compilación y Despliegue

Restaure paquetes NuGet y ejecute el servidor Kestrel:

```
dotnet restore
dotnet run --project DragonNutrex.Web
```

4. Conclusión Técnica

La implementación exitosa de esta "receta" tecnológica garantiza un sistema escalable. La separación de responsabilidades entre el Controlador y el Repositorio de Redis permite que Dragon Nutrex no solo sea una aplicación funcional, sino una plataforma robusta lista para producción. Se recomienda monitorear el uso de memoria en Redis Insight para optimizar los TTL de las sesiones de usuario.

9. Conclusión Técnica: Excelencia en el Diseño y Persistencia

La implementación de Dragon Nutrex representa un salto cuantitativo desde una aplicación de gestión local hacia una solución distribuida y de alta disponibilidad. Como desarrolladora, la adopción de la arquitectura MVC (Model-View-Controller) en conjunto con el patrón Repository no fue una elección estética, sino una decisión estratégica para garantizar el desacoplamiento total entre la lógica de negocio y la capa de datos.

¿Por qué esta arquitectura es la decisión correcta?

- **Eficiencia Extrema con Redis:** Al migrar de archivos JSON planos a una persistencia basada en Redis Cloud, hemos reducido la latencia de acceso a datos de milisegundos a microsegundos. El uso de estructuras HASH para entidades y SETS para índices de búsqueda (**menus:ids**) permite que la aplicación escale horizontalmente sin degradar el rendimiento, manejando volúmenes de datos que colapsarían un sistema de archivos tradicional.
- **Gestión Inteligente de Recursos (Singleton):** El patrón Singleton aplicado al **ConnectionMultiplexer** es el corazón de la estabilidad del sistema. Al mantener una única conexión activa y segura (Thread-Safe), optimizamos el consumo de sockets en el servidor y garantizamos que cada petición sea atendida

instantáneamente, eliminando el "overhead" de handshake de red en cada transacción.

- **Mantenibilidad y Robustez:** El estricto tipado en C# y la inyección de dependencias facilitan que el sistema sea testeable y extensible. La lógica de mapeo implementada en los repositorios asegura que, incluso si el origen de los datos cambia en el futuro, la interfaz de usuario basada en Blazor permanecerá intacta, protegiendo la inversión en el desarrollo del frontend.

El Valor del Proceso

Todo el flujo de desarrollo, desde la inyección masiva de datos mediante scripts de Python hasta el refinamiento de la reactividad en la UI, se ha centrado en crear una experiencia de usuario fluida y un backend resiliente. Hemos transformado una herramienta de seguimiento nutricional en una plataforma técnica sólida, capaz de manejar concurrencia, garantizar la integridad de los datos y ofrecer una base de código limpia que cumple con los estándares más exigentes de la industria del software actual.

Fin del Documento Técnico